Java & Computer Science

Complete Study Notes — June 2025 Batch

Volatile Memory • Variables • Conditional Statements • Relational Operators

Unit	Topic	Programs Covered
1	Volatile vs Non-Volatile Memory	—
2	Variable Declaration & Initialization	Demo.java, Intro.java, SwapNumber3rdVariable.java, AreaCircle.java
3	Conditional Statements (if/else-if/switch)	Grade.java, BMICalculator.java, ElectricityBill.java, Index.java, Bank.java, SwitchStatement.java, Nested.java, Convert.java, Tempreture.java
4	Relational Operators	All programs

UNIT 1

Volatile & Non-Volatile Memory
Computer Organization Fundamentals

1.1 What is Volatile Memory?

Definition

Volatile memory is memory that **loses all its stored data** when the power supply is turned OFF. It requires a **constant electric current** to maintain data. This type of memory is used for **temporary, fast-access storage** while programs are running.

Type	Full Name	Role	Access Speed
RAM	Random Access Memory	Main working memory for running programs	Very Fast
Cache	Cache Memory	Stores frequently used data for quicker access	Extremely Fast
Registers	CPU Registers	Tiny storage inside the CPU itself	Fastest

1.2 What is Non-Volatile Memory?

Definition

Non-volatile memory is memory that **retains its data permanently** even after the power is switched OFF. No continuous power is needed to preserve the stored information. Used for **permanent storage** of files, OS, and programs.

Type	Full Name	Role	Speed
ROM	Read Only Memory	Stores permanent boot/firmware instructions	Slow
HDD	Hard Disk Drive	Large-capacity permanent file storage	Moderate
SSD	Solid State Drive	Faster permanent storage (no moving parts)	Fast
USB	Pen Drive / USB Drive	Portable removable storage	Moderate
Card	Memory Card (SD, MicroSD)	Portable media storage for cameras, phones	Moderate

1.3 Memory Classification Diagram

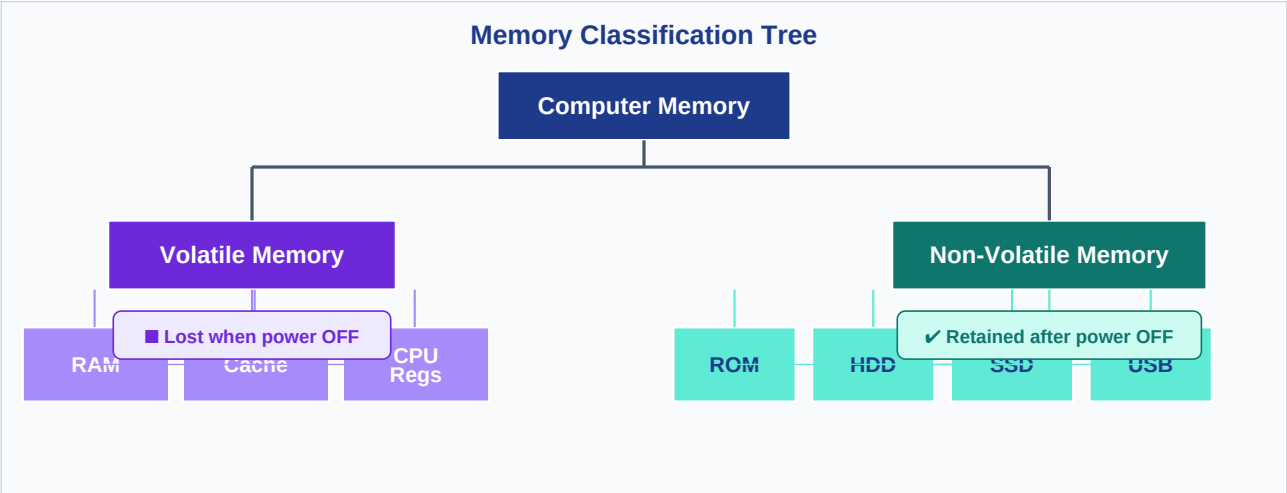

Fig 1.1 — Hierarchy of Volatile and Non-Volatile Memory

1.4 Comparison Table

Feature	Volatile Memory	Non-Volatile Memory
Power needed to keep data	■ Yes	■ No
Data after power OFF	■ Lost permanently	■ Retained safely
Speed	Faster (nanoseconds)	Slower (microseconds–ms)
Purpose	Temporary / Working storage	Permanent / Long-term storage
Cost per GB	More expensive	Less expensive
Examples	RAM, Cache, CPU Registers	ROM, HDD, SSD, USB, Memory Card

1.5 Real-Life Example

■ Scenario: You are writing a document in MS Word for 30 minutes. Suddenly the computer shuts down due to a power cut — and you had NOT pressed Ctrl+S (Save). → The unsaved content was in RAM (Volatile) → It is LOST FOREVER. → Files you had previously saved are on SSD/HDD (Non-Volatile) → Still SAFE and accessible.

■ *Memory Trick: Volatile = Temporary (Vanishes without power) | Non-Volatile = Permanent (Not erased without power)*

UNIT 2

Variable Declaration & Initialization
Java Fundamentals + 4 Programs

2.1 What is Declaration?

Definition

Declaration means **creating a variable** by specifying its *data type* and *name*. This tells Java to **reserve a memory slot** for that variable. No value is stored yet.

```
dataType variableName;

int age;        // reserves space for an integer
String name;    // reserves space for a String
double salary;  // reserves space for a decimal number
```

2.2 What is Initialization?

Definition

Initialization means **assigning a value** to a declared variable. The actual data is now written into the reserved memory location.

```
variableName = value;

age    = 22;       // stores 22 into age
name   = "Anurag"; // stores string into name
salary = 50000.0;  // stores decimal into salary
```

2.3 Combined Declaration + Initialization

You can declare and initialize in a **single line** — this is the most common approach:

```
int age         = 22;
String name     = "Anurag";
double salary   = 50000.0;
char grade      = 'A';
boolean isPass  = true;
```

2.4 Variable Lifecycle Diagram

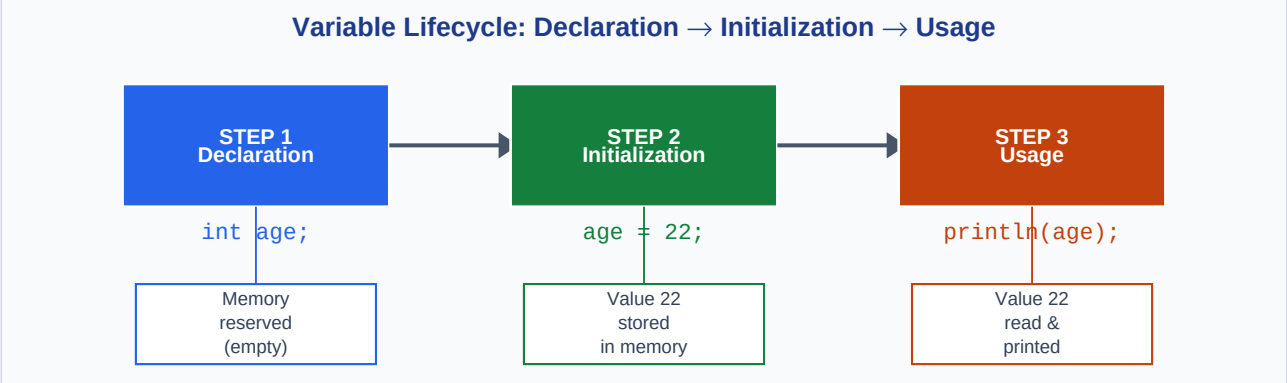

Fig 2.1 — Step-by-step lifecycle of a variable from declaration to usage

2.5 Java Data Types Quick Reference

Data Type	Size	Example Value	Use For
int	4 bytes	25, -10, 0	Whole numbers
double	8 bytes	3.14, 50000.0	Decimal numbers
char	2 bytes	'A', 'z'	Single character
String	Variable	"Anurag"	Text / words
boolean	1 bit	true / false	Yes/No conditions

2.6 Program — Demo.java

■ **Demo.java** — Demonstrates all major Java data types

```
public class Demo {
    public static void main(String[] args) {
        int age         = 25;           // integer variable
        double salary   = 50000.0;      // decimal variable
        char name       = 'A';          // single character
        String college  = "MIT University"; // text variable
        boolean isEmployed = true;      // true/false variable

        System.out.println("My name is " + name);
    }
}
```

```
■ Output: My name is A
```

■ Line-by-Line Explanation: • Each variable is declared and initialized on one line (combined form) • int, double, char, String, boolean — five different data types demonstrated • System.out.println() prints to the console with a newline at the end • String uses double quotes ("") while char uses single quotes ('')

2.7 Program — Intro.java

■ **Intro.java** — Scanner input — reading name and age from user

```
import java.util.Scanner;   // import Scanner class

public class Intro {
    public static void main(String[] args) {
        System.out.println("Enter your name:");

        Scanner input = new Scanner(System.in);  // create Scanner object

        String name = "";           // declare + initialize to empty
        name = input.next();        // read one word from user

        System.out.println("My name is " + name);

        System.out.println("Enter your age:");
        int age = 0;                // declare + initialize to 0
        age = input.nextInt();      // read integer from user

        System.out.println("My age is " + age);
    }
}
```

```
■ Output: Enter your name:
Anurag
My name is Anurag
Enter your age:
21
My age is 21
```

■ Key Concepts: • import java.util.Scanner — must import to use Scanner • new Scanner(System.in) — creates a Scanner that reads from keyboard • input.next() — reads one word (stops at space) • input.nextInt() — reads an integer number • Variables are first declared with default values, then assigned from input

2.8 Program — SwapNumber3rdVariable.java

■ **SwapNumber3rdVariable.java** — Swap two variables using a temporary variable

```
public class SwapNumber3rdVariable {
    public static void main(String[] args) {
        int a = 10, b = 20;           // initialize both variables

        System.out.println("Before Swap: a = " + a + ", b = " + b);

        int temp = a;   // Step 1: save 'a' in temp  → temp=10
        a = b;          // Step 2: copy 'b' into 'a' → a=20
        b = temp;       // Step 3: copy temp into 'b' → b=10

        System.out.println("After Swap:  a = " + a + ", b = " + b);
    }
}
```

```
■ Output: Before Swap: a = 10, b = 20
After Swap:  a = 20, b = 10
```

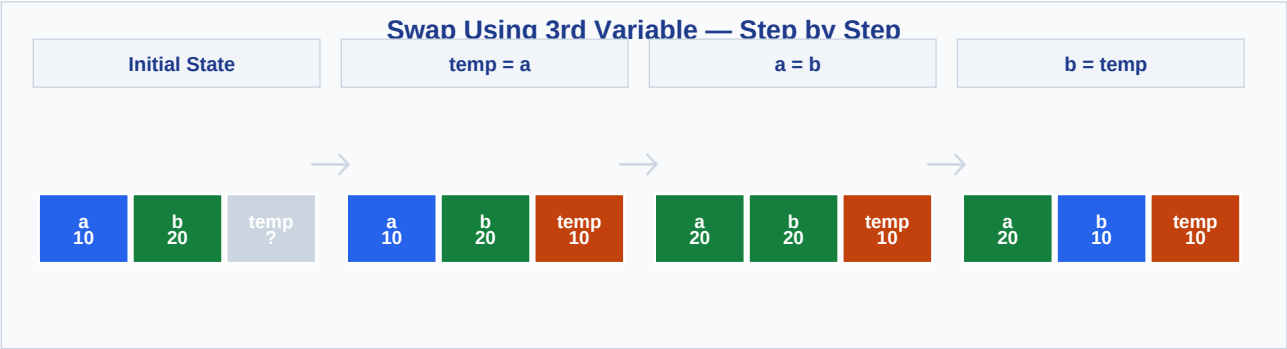

Fig 2.2 — Memory state at each step of the swap operation

■ Why do we need 'temp'? If we directly do a = b, the original value of a (10) is gone forever. temp acts like a safe holding spot. Think of it like: you have tea in cup A and coffee in cup B. You need an empty cup C (temp) to swap them without mixing!

2.9 Program — AreaCircle.java

■ **AreaCircle.java** — Calculate area of a circle using Scanner input

```
import java.util.Scanner;

public class AreaCircle {
    public static void main(String[] args) {
        int radius = 0;                      // declare radius
```

```
        Scanner sc = new Scanner(System.in); // create Scanner

        radius = sc.nextInt();               // read radius from user

        // Formula: Area = π × r²   (using 3.14 as π)
        System.out.println("Area of Circle is: " + (3.14 * radius * radius));
    }
}
```

```
■ Output: 7
Area of Circle is: 153.86
```

■ Key Concepts: • 3.14 * radius * radius applies the formula π×r² • int radius is converted to double automatically when multiplied by 3.14 (type promotion) • Tip: Use Math.PI instead of 3.14 for higher precision in real projects

UNIT 3

Conditional Statements
if / if-else / else-if / switch / Nested — 9 Programs

3.1 Theory — All Conditional Statements

Statement	When to Use	Structure
if	Single condition check	if (cond) { }
if-else	Two-way decision (true or false)	if (cond) { } else { }
else-if	Multiple sequential conditions	if { } else if { } else { }
switch	Match against fixed constant values	switch(val) { case x: break; }
Nested if	Conditions inside other conditions	if { if { if { } } }

3.2 Flowchart — if / else-if / else

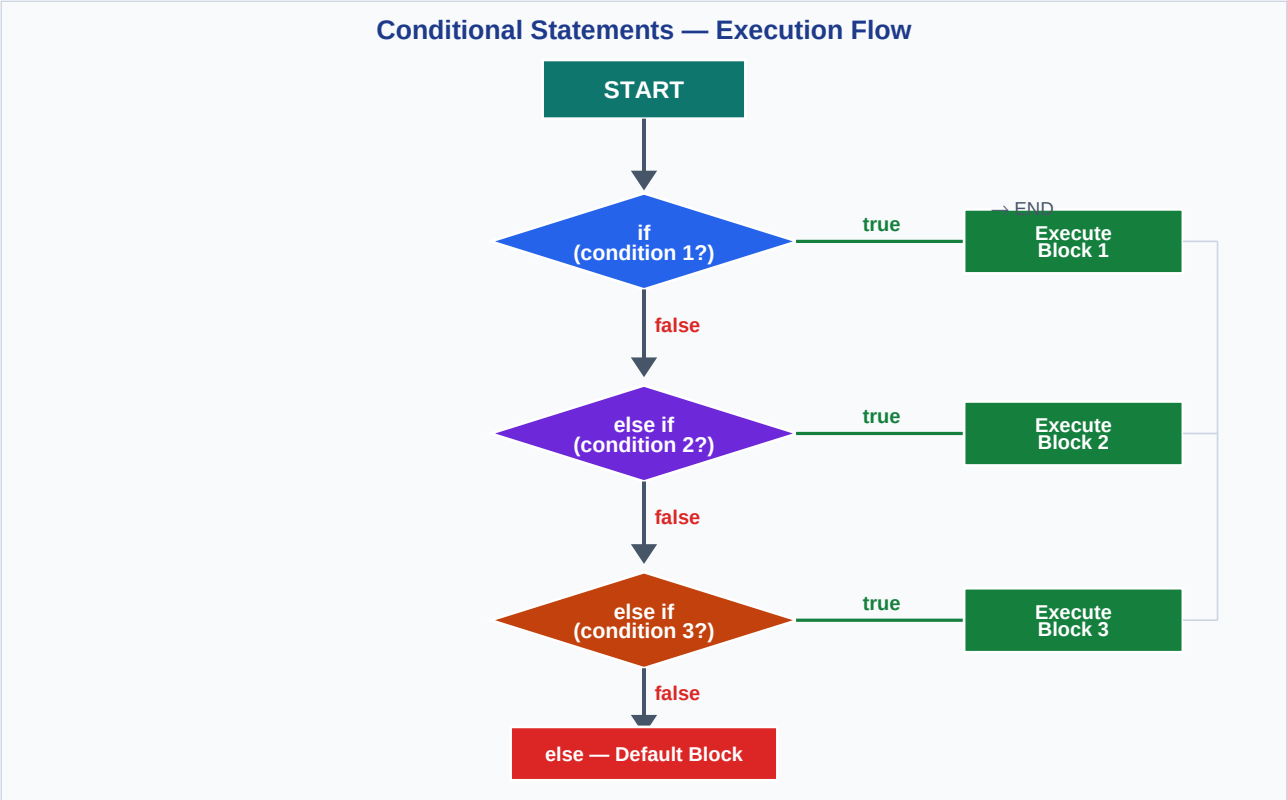

Fig 3.1 — Execution flow: only the FIRST true condition block runs, then control exits

3.3 Flowchart — switch Statement

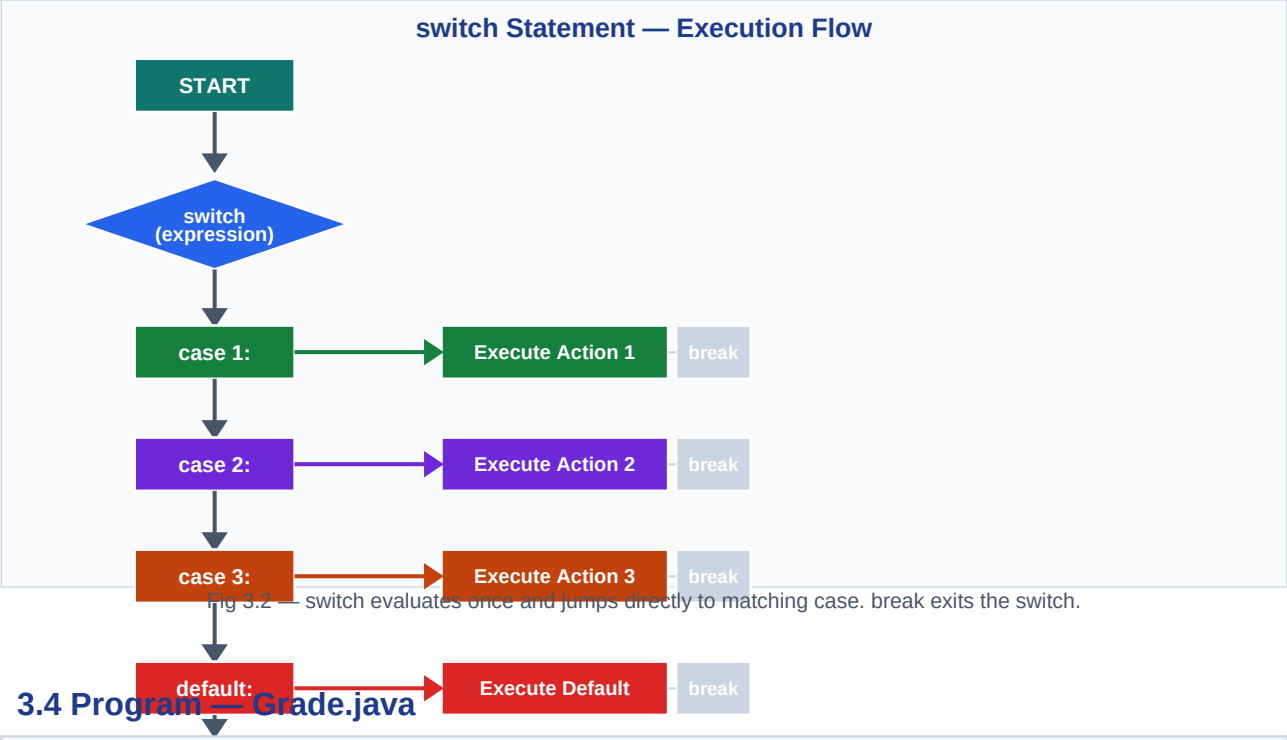

Fig 3.2 — switch evaluates once and jumps directly to matching case. break exits the switch.

3.4 Program — Grade.java

■ **Grade.java** — Assign letter grade based on marks using else-if ladder

```
package Day1.Conditional_Statements;

public class Grade {
    public static void main(String[] args) {
        int marks = 85;

        if (marks >= 90 && marks <= 100) {       // condition 1
            System.out.println("Grade: A");
        }
        else if (marks >= 80 && marks < 90) {    // condition 2 — matches!
            System.out.println("Grade: B");
        }
        else if (marks >= 70 && marks < 80) {    // condition 3
            System.out.println("Grade: C");
        }
        else if (marks >= 60 && marks < 70) {    // condition 4
            System.out.println("Grade: D");
        }
        else {                                   // none matched
            System.out.println("Grade: F");
        }
    }
}
```

■ `Output: Grade: B`

■ Explanation: • marks = 85 → condition 1 (>=90): false → condition 2 (>=80 && <90): TRUE → prints 'Grade: B' • && means BOTH conditions must be true (AND operator — covered in Logical Operators) • Once a match is found, remaining else-if blocks are SKIPPED

3.5 Program — BMICalculator.java

■ **BMICalculator.java** — Calculate BMI and classify health category

```
package Day1.Conditional_Statements;

public class BMICalculator {
    public static void main(String[] args) {
        double weight = 55.00;   // in kg
        double height = 1.65;    // in metres

        double bmi = weight / (height * height);  // BMI formula

        if (bmi < 18.5) {
            System.out.println("Chal be Patlu!");           // Underweight
        }
        else if (bmi >= 18.5 && bmi < 25) {
            System.out.println("Tu toh fit hai!!");         // Normal
        }
        else if (bmi >= 25 && bmi < 30) {
            System.out.println("Chal be Motte!");           // Overweight
        }
        else {
            System.out.println("Obese Category");           // Obese
        }
    }
}
```

```
■ Output: Tu toh fit hai!!
```

BMI Calculation: 55 ÷ (1.65 × 1.65) = 55 ÷ 2.7225 ≈ 20.2 → falls in 18.5–25 range → Normal

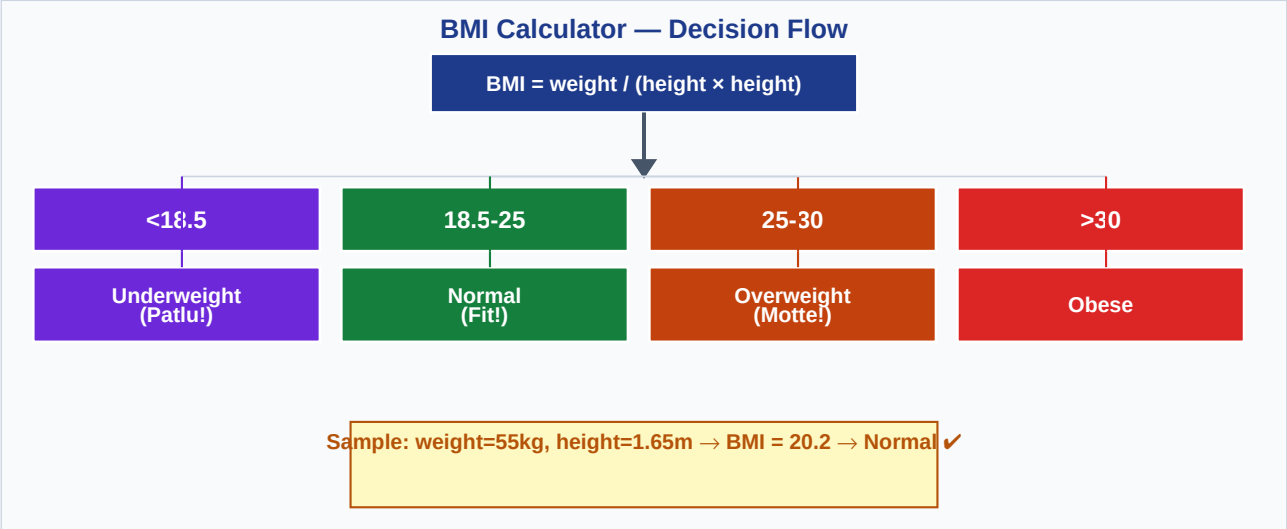

Fig 3.3 — BMI classification decision diagram

3.6 Program — ElectricityBill.java

■ **ElectricityBill.java** — Slab-rate electricity billing system

```
package Day1.Conditional_Statements;

public class ElectricityBill {
    public static void main(String[] args) {
        int units = 320;         // units consumed
        int billAmount = 0;

        if (units <= 100) {
            billAmount = units * 1;                              // ■1/unit
```

```
        }
        else if (units > 100 && units <= 200) {
            billAmount = 100*1 + (units-100)*2;                    // ■2/unit after 10
0
        }
        else if (units > 200 && units <= 300) {
            billAmount = 100*1 + 100*2 + (units-200)*3;            // ■3/unit after 200
        }
        else {
            billAmount = 100*1 + 100*2 + 100*3 + (units-300)*4;    // ■4/unit after 300
        }
        System.out.println("Bill Amount: " + billAmount);
    }
}
```

```
■ Output: Bill Amount: 680
```

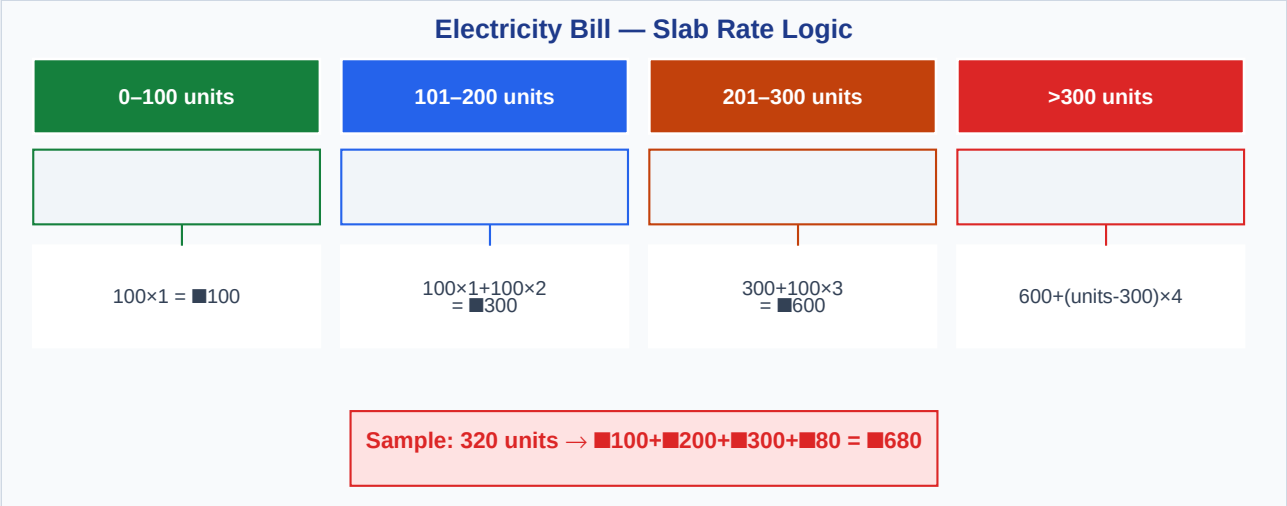

Fig 3.4 — Slab rate electricity billing logic

■ Calculation Trace for 320 units: First 100 units → 100 × ■1 = ■100 Next 100 units → 100 × ■2 = ■200 Next 100 units → 100 × ■3 = ■300 Remaining 20 units → 20 × ■4 = ■80 Total = ■100 + ■200 + ■300 + ■80 = ■680

3.7 Program — Index.java

■ **Index.java** — ATM withdrawal — simple if-else balance check

```
package Day1.Conditional_Statements;
import java.util.Scanner;

public class Index {
    public static void main(String[] args) {
        int balance = 1000;

        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the amount to withdraw: ");

        int amt = sc.nextInt();

        if (balance >= amt) {                          // sufficient balance?
            System.out.println(amt + " Amt debited - avl balance is " + (balance - amt))
;
```

```
        } else {
            System.out.println("Chal bee Gareeb!!!!");  // insufficient balance
        }
    }
}
```

```
■ Output: Enter the amount to withdraw:
500
500 Amt debited - avl balance is 500
```

■ Concept: Simple if-else for two outcomes — either you have enough balance (debit) or you don't (error message). The relational operator >= (greater than or equal to) is the key check here.

3.8 Program — Bank.java

■ **Bank.java** Full banking menu using switch + nested if

```
package Day1.Conditional_Statements;
import java.util.Scanner;

public class Bank {
    public static void main(String[] args) {
        System.err.println("1.Withdrawal  2.Deposit  3.CurrentBalance  4.FD");
        int balance = 2000;
        Scanner obj = new Scanner(System.in);
        int option = obj.nextInt();

        switch (option) {
            case 1:                                    // Withdrawal
                System.out.println("Enter the amt to withdraw");
                int amt = obj.nextInt();
                if (balance > amt) {                   // nested if inside switch
                    balance -= amt;
                    System.out.println(amt + " is debited from your account");
                }
                break;

            case 2:                                    // Deposit
                System.out.println("Enter the amt to deposit");
                int amtToDeposit = obj.nextInt();
                balance += amtToDeposit;
                break;

            case 3:                                    // Balance
                System.out.println(balance);
                break;

            case 4:                                    // Fixed Deposit
                System.out.println("Enter the time (years):");
                int time = obj.nextInt();
                System.out.println("Amount after " + time + " years: " + (balance * 7 *
time) / 100);
                break;

            default:
                System.out.println("Abe andha hai kyaaa!!!");
                break;
```

```
        }
    }
}
```

```
■ Output: 1.Withdrawal  2.Deposit  3.CurrentBalance  4.FD
3
2000
```

■ Key Concepts Used: • switch(option) — directs to correct banking operation based on user's choice • case 1 has a nested if — checks balance BEFORE allowing withdrawal • balance -= amt is shorthand for balance = balance - amt • break is MANDATORY — without it, execution falls through to next case • default handles invalid inputs gracefully

3.9 Program — SwitchStatement.java (Fall-Through Demo)

■ **SwitchStatement.java** — Food ordering menu — demonstrates switch fall-through bug

```
package Day1.Conditional_Statements;
import java.util.Scanner;

public class SwitchStatement {
    public static void main(String[] args) {
        System.err.println("1.Parantha  2.Chai  3.Sandwich  4.Hwa Khanni Hai");

        Scanner obj = new Scanner(System.in);
        int option = obj.nextInt();

        switch (option) {
            case 1:
                System.out.println("Lo ji parantha");
                // ■■  NO break here! — falls through to case 2
            case 2:
                System.out.println("Lo ji chai");
                break;            // stops here
            case 3:
                System.out.println("Lo ji sandwich");
                break;
            case 4:
                System.out.println("Hwa khanni hai toh side mai baith");
                break;
            default:
                System.out.println("Abe andha hai kyaaa!!!");
                break;
        }
    }
}
```

```
■ Output: If input = 1:
Lo ji parantha
Lo ji chai       ← BOTH print due to missing break in case 1!
```

■■ Important — switch Fall-Through

Case 1 is **missing the break statement**. When option=1 is selected, it prints 'Lo ji parantha' AND then falls through and prints 'Lo ji chai' too. This is a common bug — **always add break** unless fall-through is intentional.

3.10 Program — Nested.java

■ **Nested.java** — Placement eligibility checker — nested if conditions

```
package Day1.Conditional_Statements;

public class Nested {
    public static void main(String[] args) {
        int tenth     = 80;
        int twelth    = 65;
        int CG        = 5;       // ■■  CG < 7 — will fail here
        int placement = 85;

        if (tenth > 70) {                    // LEVEL 1: 10th check
            if (twelth > 60) {               // LEVEL 2: 12th check
                if (CG > 7) {                // LEVEL 3: CGPA check — FAILS (5 > 7 = f
alse)
                    if (placement > 80) {    // LEVEL 4: placement
                        System.out.println("Placement crack!!!");
                    } else {
                        System.out.println("Chal betta bahar placement no nhi ayye");
                    }
                } else {
                    System.out.println("Chal betta bahar CG kaam hai"); // ← executes
                }
            } else {
                System.out.println("Chal betta bahar 12 mai nhi hai");
            }
        } else {
            System.out.println("Chal betta bahar 10 mai no nhi hai");
        }
    }
}
```

■ Output: `Chal betta bahar CG kaam hai`

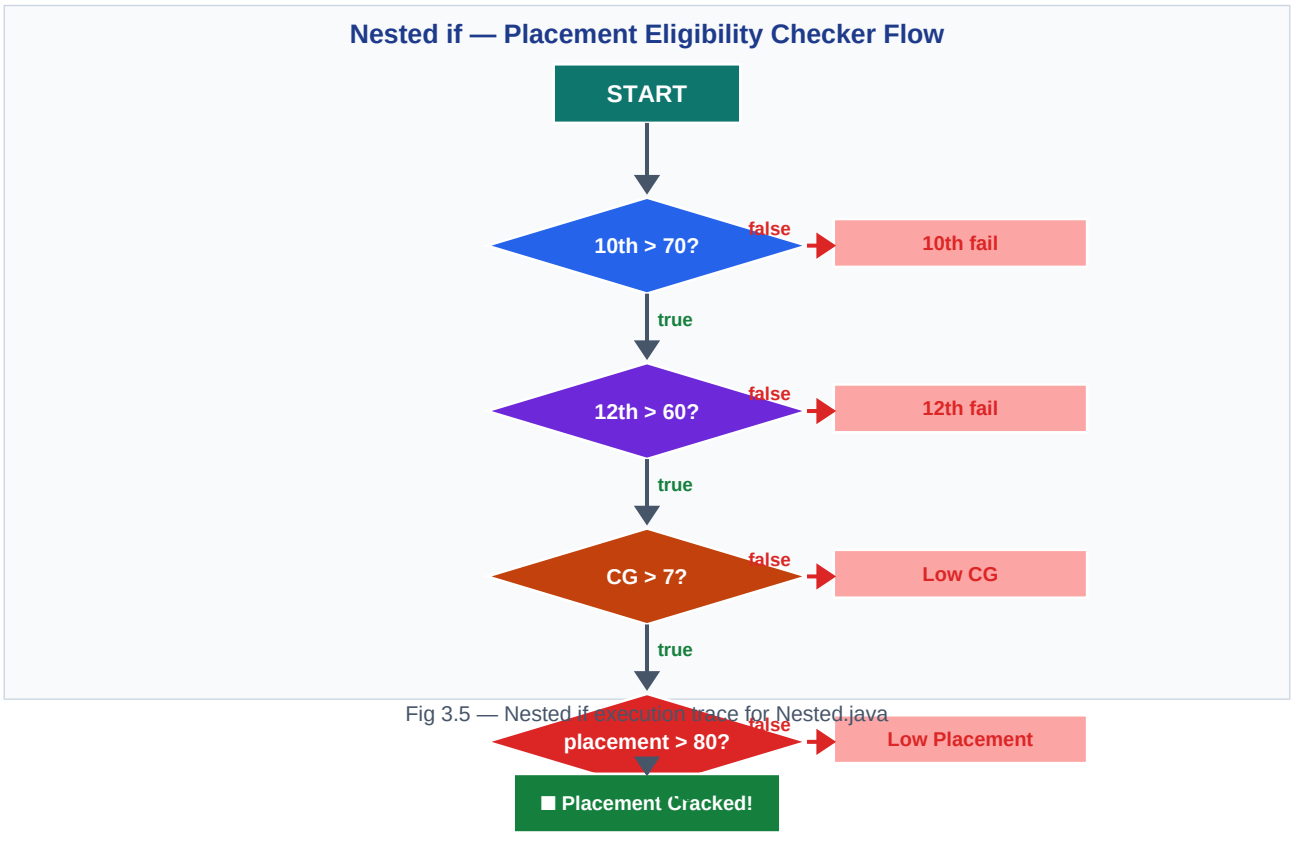

Fig 3.5 — Nested if execution trace for Nested.java

■ Trace: tenth=80 > 70 ✔ → twelth=65 > 60 ✔ → CG=5 > 7 ✗ → prints 'CG kaam hai' Nested ifs are like security checkpoints — ALL must pass to reach the final success.

3.11 Program — Convert.java

■ **Convert.java** Temperature unit converter direction selector

```
package Day1.Conditional_Statements;
import java.util.Scanner;

public class Convert {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        char symbol = sc.next().charAt(0);    // read first character of input

        if (symbol == 'f') {
            System.out.println("Take input in Celsius");    // converting F→C
        } else if (symbol == 'c') {
            System.out.println("Take input in Fahrenheit"); // converting C→F
        } else {
            System.out.println("Invalid symbol entered");
        }
    }
}
```

```
■ Output: f
Take input in Celsius
```

■ Key Concept: sc.next().charAt(0) — reads a full word but takes only the first character. char comparison uses == with single quotes ('f'). This program sets the direction — the actual conversion is in Tempreture.java.

3.12 Program — Tempreture.java

■ **Tempreture.java** Celsius to Fahrenheit converter

```
package Day1.Conditional_Statements;
import java.util.Scanner;

public class Tempreture {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Temperature in Celsius: ");
        int celsius = sc.nextInt();

        // Formula: F = (C × 9/5) + 32
        double fahrenheit = (celsius * 9/5) + 32;

        System.out.println("Temperature in Fahrenheit: " + fahrenheit);
    }
}
```

```
■ Output: Enter Temperature in Celsius: 100
Temperature in Fahrenheit: 212.0
```

■■ Common Pitfall in This Code

The expression **celsius * 9/5** performs **integer division** because 9 and 5 are both integers. For 37°C: 37×9=333, 333/5=66 (not 66.6). Fix: use **9.0/5** for accurate decimal result.

```
// Fix:  double fahrenheit = (celsius * 9.0/5) + 32;
```

UNIT 4

Relational Operators in Java

Compare values → return true or false

4.1 What are Relational Operators?

Definition

Relational operators **compare two values** and always return a **boolean result** — either **true** or **false**. They are primarily used inside *if, else-if, while, and for* conditions.

Fig 4.1 — All six relational operators with their purpose

Operator	Meaning	Example (a=10, b=5)	Result
==	Equal to	a == b → 10 == 5	false
!=	Not equal to	a != b → 10 != 5	true
>	Greater than	a > b → 10 > 5	true
<	Less than	a < b → 10 < 5	false
>=	Greater than or equal	a >= b → 10 >= 5	true
<=	Less than or equal	a <= b → 10 <= 5	false

4.2 Code Examples

```
int a = 10, b = 5;

System.out.println(a == b);   // false — 10 is NOT equal to 5
System.out.println(a != b);   // true  — 10 IS different from 5
System.out.println(a >  b);   // true  — 10 is greater than 5
System.out.println(a <  b);   // false — 10 is NOT less than 5
System.out.println(a >= b);   // true  — 10 is greater than 5
System.out.println(a <= b);   // false — 10 is NOT less than or equal to 5
```

```
■ Output: false
true
true
false
true
false
```

4.3 Relational Operators in Your Programs

Program	Operator Used	Where / Purpose
Grade.java	>=, <	marks >= 90, marks < 90 — range check for grades
BMICalculator.java	<, >=	bmi < 18.5, bmi >= 18.5 — BMI category check
ElectricityBill.java	<=, >	units <= 100, units > 100 — slab boundary check
Index.java	>=	balance >= amt — sufficient balance check
Bank.java	>	balance > amt — withdrawal condition
Nested.java	>	tenth > 70, CG > 7 — eligibility thresholds
Tempreture.java	(used in extension)	celsius comparison for direction
SwapNumber3rdVariable.java	==	verifying swap (conceptually)

■ *Memory Trick: E – N – G – L – GE – LE = Equal (==) • Not Equal (!=) • Greater (>) • Less (<) • Greater-Equal (>=) • Less-Equal (<=)*

■ Quick Revision Summary — All Units

Unit	Topic	Key Point	Remember
1	Volatile Memory	Loses data on power OFF	RAM, Cache, Registers
1	Non-Volatile Memory	Keeps data without power	ROM, HDD, SSD, USB
2	Declaration	Create variable — reserve memory	int age;
2	Initialization	Assign value to variable	age = 22;
2	Scanner	Read input from keyboard	new Scanner(System.in)
2	Swap (3rd var)	Use temp to hold one value	temp=a; a=b; b=temp;
3	if	One condition	if (cond) { }
3	if-else	Two-way decision	if { } else { }
3	else-if	Multiple conditions	else if (cond) { }
3	switch	Fixed value matching	case x: ... break;
3	Nested if	Conditions inside conditions	if { if { if { } } }
3	Fall-through	Missing break in switch	Always add break!
4	== (Equal)	Check if values are same	10 == 10 → true
4	!= (Not Equal)	Check if values differ	10 != 5 → true
4	> < >= <=	Compare numeric values	10 > 5 → true

End of Notes — All the best! ■